PLC-Based Sample PLC/ICS Project

OpenPLC Temperature-Control System

Jason Cala | August 2026 | github.com/jasoncala/openplc-temperature-control

Project Summary

This project completes the CPSec & CyFI Labs exploratory assignment by implementing a software-only temperature-control loop using OpenPLC Editor, Structured Text, OpenPLC Runtime v3, Docker, Modbus TCP, Python, and PyModbus. Python simulates the temperature sensor, OpenPLC executes the fan logic, and Python reads the resulting fan output.

Temperature condition	Expected fan
>= 30 C	ON
<= 27 C	OFF
28-29 C	Retain previous state

The separate 30 C ON threshold and 27 C OFF threshold create a 3 C hysteresis band. The final ordered validation passed 9 of 9 cases, including both threshold boundaries and both retained-state cases.

FINAL RESULT: 9 TESTS PASSED, 0 FAILED

Thoughts

Overall, I greatly enjoyed the exploratory work that I tackled in this simple project. I would love to continue with this learning and pursue deeper, cutting-edge research in ICS security, PLC applications, and AI/ML. I would love to hear more about the opportunities and projects available.

Report Sections

Assignment task	Main evidence
1. Setup OpenPLC	Screenshots 01-09; Dockerfile; Runtime notes
2. Learn Structured Text	Boolean project; screenshots 10-14
3. Define scenario	Controller project; screenshots 15-19
4. Deploy and execute	Runtime upload and Running status; screenshots 20-21
5. Establish Modbus	PyModbus client; screenshots 22-24
6. Validate results	Nine-test CSV, console output, plot; screenshots 25-26

Assignment Task 1: Explore and Setup OpenPLC

Install OpenPLC editor and Runtime. Document successful setup.

Steps Performed:

1. Recorded Windows, virtualization, WSL, Git, Python, Docker, and VS Code details.

2. Installed and opened OpenPLC Editor.

- Chose Runtime v3 because its browser dashboard and Programs upload page directly matched the assignment's web-interface requirement. V3 is no longer maintained (archived repo), so for that reason, it was pinned, containerized, restricted to localhost, and used only for this research exercise.

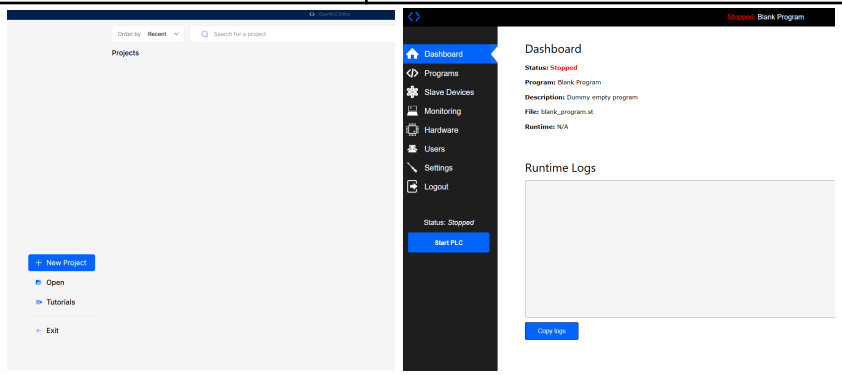
3. Pinned the Runtime source to commit b5d41356dab4aeadca0dd7ca64ba542f870b595d.

4. Built openplc-v3-local with the custom Dockerfile and created the persistent volume openplc-v3-data.

5. Started container openplc-v3 with dashboard port 8080 and Modbus port 1502 bound only to 127.0.0.1.

6. Logged in through the Runtime web interface, changed the default password, and verified the new password.

Recorded component	Value
Host	Windows 11 Home 25H2
Python	3.11.9
Docker Desktop	4.84.0
Docker Engine / CLI	29.6.2
VS Code	1.108.1
Runtime endpoint	http://127.0.0.1:8080
Modbus endpoint	127.0.0.1:1502



Figures 1-4. Editor installation, Runtime dashboard, Docker image, and localhost port evidence

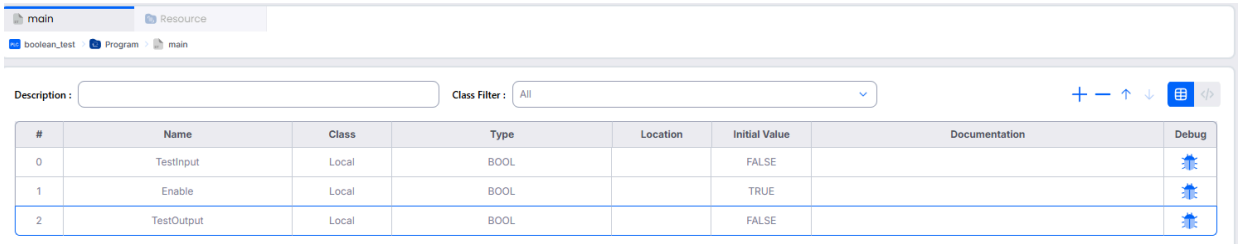
Assignment Task 2: Learn PLC Programming with Structured Text

Write a small Boolean program and verify the Editor and simulator environment.

Steps Performed:

1. Created plc/boolean_test in OpenPLC Editor.

2. Added TestInput, Enable, and TestOutput as Boolean variables

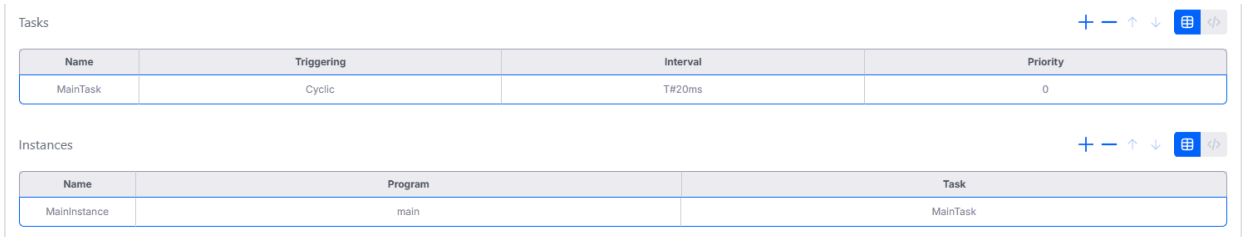


#	Name	Class	Type	Location	Initial Value	Documentation	Debug
0	TestInput	Local	BOOL		FALSE		
1	Enable	Local	BOOL		TRUE		
2	TestOutput	Local	BOOL		FALSE		

3. Implemented the AND expression shown below.

```
1    TestOutput := TestInput AND Enable;
```

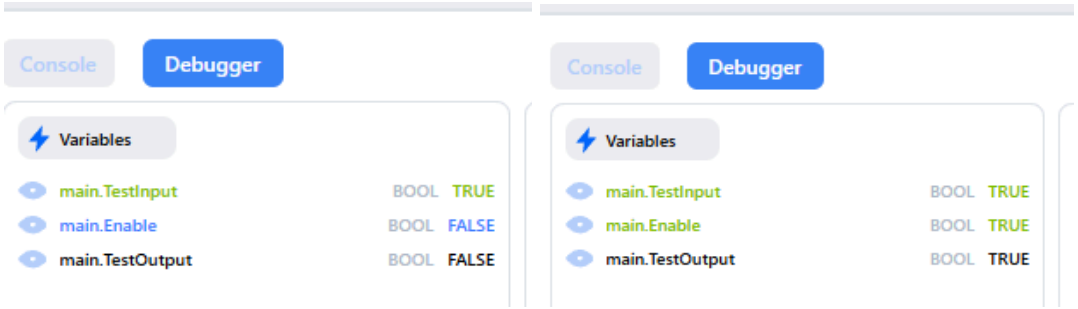
4. Configured a cyclic MainTask and MainInstance.



Name	Triggering	Interval	Priority
MainTask	Cyclic	T#20ms	0

Name	Program	Task
MainInstance	main	MainTask

5. Built and ran the project in the simulator.



6. Tested all four Boolean combinations; all produced the expected output.

Full Results:

TestOutput := TestInput AND Enable;

TestInput	Enable	Expected	Result
FALSE	FALSE	FALSE	PASS
FALSE	TRUE	FALSE	PASS
TRUE	FALSE	FALSE	PASS
TRUE	TRUE	TRUE	PASS

Assignment Task 3: Define the Temperature-Control Scenario

Define the simulated sensor, fan output, thresholds, logic, and rationale.

Steps Performed:

1. Defined variables:

- Temperature as UINT at %MW0 with initial value 20.
- Fan as BOOL at %QX0.0 with initial value FALSE.

#	Name	Class	Type	Location	Initial Value	Documentation	Debug
0	Temperature	Local	UINT	%MW0	20		
1	Fan	Local	BOOL	%QX0.0	FALSE		

2. Selected 30 C as the ON threshold and 27 C as the OFF threshold.

- Used a 3 C hysteresis band so the middle range preserves the prior state.

```
(* Cooling fan control with 3-degree hysteresis. *)

IF Temperature >= 30 THEN
    Fan := TRUE;
ELSIF Temperature <= 27 THEN
    Fan := FALSE;
END_IF;
```

4. Implemented the controller in plc/temperature_control.

5. Configured a 20 ms cyclic task.

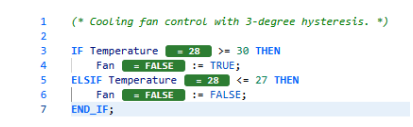
Tasks

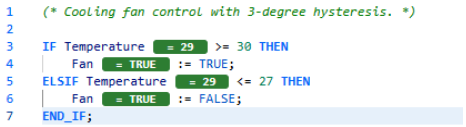
Name	Triggering	Interval	Priority
MainTask	Cyclic	T#20ms	0

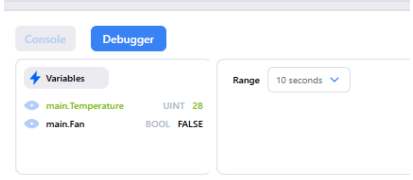
Instances

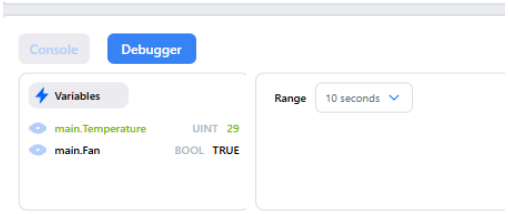
Name	Program	Task
MainInstance	main	MainTask

6. Confirmed in simulation that 28 C retained OFF and 29 C retained ON.









Assignment Task 4: Deploy and Execute the Control Logic

Upload the Structured Text program to Runtime and confirm operation in the web interface.

Steps Performed:

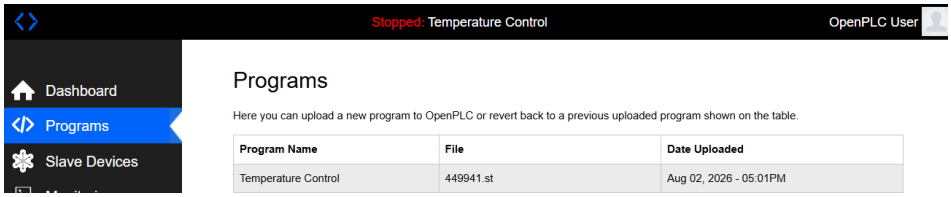
1. Created the complete Runtime file plc/exported/temperature_control_runtime.st.

- This included the program, located variables, hysteresis logic, configuration, resource, cyclic task, and program instance.

```
PROGRAM main
VAR
   Temperature AT %MW0 : UINT := 20;
   Fan AT %QX0.0 : BOOL := FALSE;
END_VAR

(* control logic omitted here for space *)

TASK MainTask(INTERVAL := T#20ms, PRIORITY := 0);
PROGRAM MainInstance WITH MainTask : main;
```

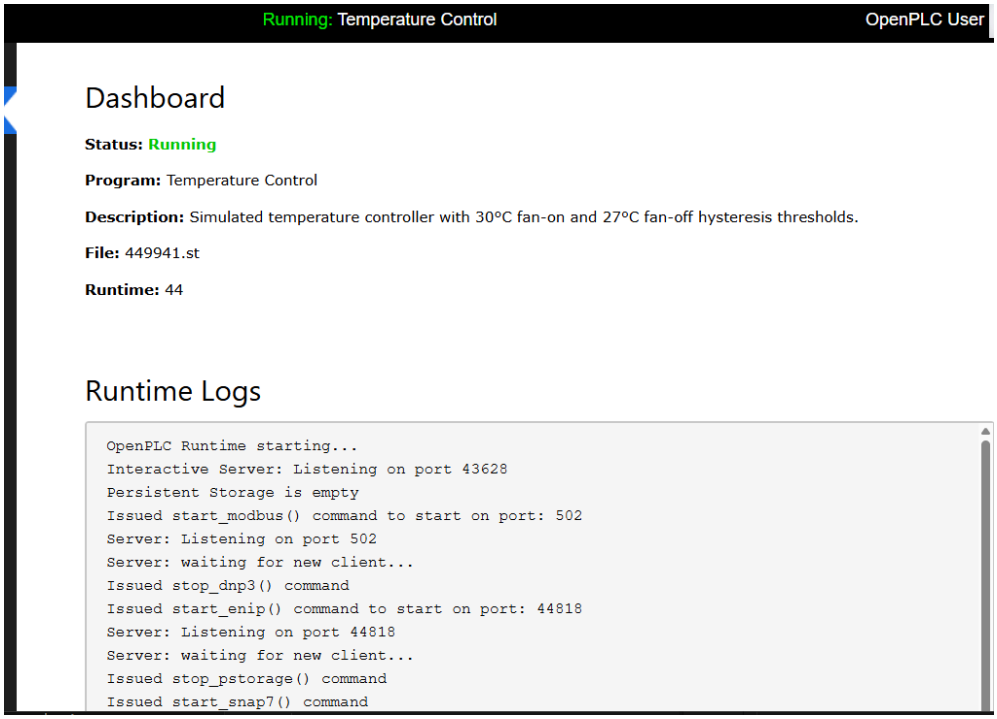
2. Opened the Runtime dashboard at http://127.0.0.1:8080.

3. Uploaded the .st file through the Programs page as Temperature Control.



4. Waited for compilation and reviewed the Runtime log.

5. Confirmed the dashboard displayed the program in the Running state.



Assignment Task 5: Establish Modbus Communication

Use PyModbus to send simulated temperature data and read the fan control signal.

Steps Performed:

1. Created a Python 3.11 virtual environment and installed PyModbus 3.11.1.

2. Confirmed host port 1502 forwarded to OpenPLC container port 502.

```
PS C:\Users\jason\Documents\plc-temperature-control> Test-NetConnection `
>>     -ComputerName 127.0.0.1 `
>>     -Port 1502


ComputerName     : 127.0.0.1
RemoteAddress    : 127.0.0.1
RemotePort       : 1502
InterfaceAlias   : Loopback Pseudo-Interface 1
SourceAddress    : 127.0.0.1
TcpTestSucceeded : True
```

3. Mapped %MW0 to holding register 1024 and %QX0.0 to coil 0.

PLC variable	IEC location	Modbus area	Address
Temperature	%MW0	Holding register	1024
Fan	%QX0.0	Coil	0

4. Created client/modbus_smoke_test.py with connection, write, read, response-checking, timeout, and cleanup logic.

5. Wrote the requested temperature, waited for PLC scans, read the register back, and read the fan coil.

6. Confirmed 20 C produced fan OFF and 35 C produced fan ON.

```
(.venv) PS C:\Users\jason\Documents\plc-temperature-control> python client\modbus_smoke_test.py `
>>     --temperature 20 `
>>     2>&1 |
>>     Tee-Object `
>>         -FilePath "results\smoke_test_20.txt"
Connected to: 127.0.0.1:1502
Temperature written: 20
Temperature read: 20
Fan state: OFF
```

```
(.venv) PS C:\Users\jason\Documents\plc-temperature-control> python client\modbus_smoke_test.py `
>>     --temperature 35 `
>>     2>&1 |
>>     Tee-Object `
>>         -FilePath "results\smoke_test_35.txt"
Connected to: 127.0.0.1:1502
Temperature written: 35
Temperature read: 35
Fan state: ON
```

Assignment Task 6: Validate and Document the Results

Vary sensor inputs, compare actual and expected behavior, and document the results.

Steps Performed:

1. Created client/validate_controller.py.

2. Reset the PLC to a deterministic low-temperature OFF state.

3. Applied the ordered sequence 20, 27, 28, 30, 29, 27, 40, 35, and 27 C.

4. Calculated the expected fan state independently in Python.

5. Polled the Runtime until the expected temperature and fan state appeared or the timeout expired.

6. Recorded written val, read val, expected fan, actual fan, elapsed time, and PASS/FAIL result.

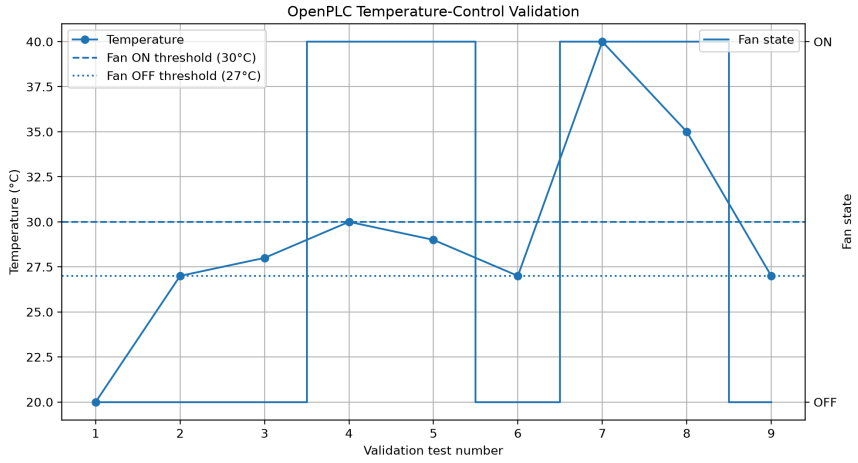
```
(.venv) PS C:\Users\jason\Documents\plc-temperature-control> python client\validate_controller.py `
>>     2>&1 |
>>     Tee-Object `
>>         -FilePath "results\validation_console.txt"
PASS | Test 1 | Establish known low state | Written=20 | Read=20 | Expected fan=OFF | Actual fan=OFF | Elapsed=0.000s
PASS | Test 2 | Exact fan-off threshold | Written=27 | Read=27 | Expected fan=OFF | Actual fan=OFF | Elapsed=0.000s
PASS | Test 3 | Hysteresis region retains OFF | Written=28 | Read=28 | Expected fan=OFF | Actual fan=OFF | Elapsed=0.000
s
PASS | Test 4 | Exact fan-on threshold | Written=30 | Read=30 | Expected fan=ON | Actual fan=ON | Elapsed=0.110s
PASS | Test 5 | Hysteresis region retains ON | Written=29 | Read=29 | Expected fan=ON | Actual fan=ON | Elapsed=0.000s
PASS | Test 6 | Downward transition | Written=27 | Read=27 | Expected fan=OFF | Actual fan=OFF | Elapsed=0.109s
PASS | Test 7 | Clearly high temperature | Written=40 | Read=40 | Expected fan=ON | Actual fan=ON | Elapsed=0.109s
PASS | Test 8 | Remain ON above threshold | Written=35 | Read=35 | Expected fan=ON | Actual fan=ON | Elapsed=0.000s
PASS | Test 9 | Final shutdown | Written=27 | Read=27 | Expected fan=OFF | Actual fan=OFF | Elapsed=0.094s

Tests completed: 9
Tests passed: 9
Tests failed: 0
Results saved to: results\validation_results.csv
```

7. Saved results/validation_results.csv and generated results/validation_plot.png.

validate_controller.py U | validation_results.csv U | modbus_smoke_test.py | modbus_map.md | temperature_control_runtime.st

sults > validation_results.csv

```
1  test_number,description,temperature_written,temperature_read,previous_expected_fan,expected_fan,actual_fan,elapsed_seconds,passed
2  1,Establish known low state,20,20,False,False,False,0.0,True
3  2,Exact fan-off threshold,27,27,False,False,False,0.0,True
4  3,Hysteresis region retains OFF,28,28,False,False,False,0.0,True
5  4,Exact fan-on threshold,30,30,False,True,True,0.11,True
6  5,Hysteresis region retains ON,29,29,True,True,True,0.0,True
7  6,Downward transition,27,27,True,False,False,0.109,True
8  7,Clearly high temperature,40,40,False,True,True,0.109,True
9  8,Remain ON above threshold,35,35,True,True,True,0.0,True
10 9,Final shutdown,27,27,True,False,False,0.094,True
11
```



Conclusion

As written in the report above as well as shown in the github repository (https://github.com/jasoncala/openplc-temperature-control), I completed all six requested assignment tasks. This report demonstrates OpenPLC setup, Structured Text programming, a documented temperature-control scenario, deployment through the Runtime web interface, Modbus TCP communication from Python, and repeatable validation. All nine ordered validation cases passed, including exact thresholds and both hysteresis-retention cases.

This assignment has only increased my interest in the CPSec lab further. It matched my interest of software involved with physical systems and I believe further investigating the ways AI and security niches interact with these things would be fascinating to me. I believe my foundations and my commitment would make me a good candidate for the laboratory as a long term investment. I am committed to growth, learning, and delivering results even through hardship.

Repository Evidence & Reproduction
Throughout the repository there are many pieces of evidence I've uploaded including screenshots, notes as markdown files, and logs as txt files.

If the project is needed to be reproduced then the repo's README file will be helpful. For example the following steps could be followed in windows powershell:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\scripts\start_openplc.ps1
.\scripts\status_openplc.ps1

# After uploading the Runtime ST file:
.\.venv\Scripts\Activate.ps1
python client\modbus_smoke_test.py --temperature 20
python client\modbus_smoke_test.py --temperature 35
python client\validate_controller.py
python client\plot_results.py
```